

IndiNews Search Engine

Building and Evaluating a Mini Web Search Engine
with Cross-Lingual Retrieval

Course	Information Retrieval — S2-25_AIMLZG537
Group	Group 37
Domain	Indian News & Current Affairs
Language	English ↔ Hindi (Cross-Lingual)
UI	Streamlit

1. Introduction

This report documents the design, implementation, and evaluation of **IndiNews** — a domain-specific mini web search engine built over a bilingual (English–Hindi) corpus of Indian news and current affairs. The system addresses all mandatory and optional requirements of Assignment #2:

- Ranked retrieval using the Vector Space Model (TF-IDF + cosine similarity)
- Text mining enhancement via Latent Dirichlet Allocation (LDA) topic modelling for re-ranking
- Cross-lingual retrieval using the query translation approach (deep-translator) supporting Hindi ↔ English bidirectional search
- Rigorous evaluation on 25 manually judged queries using Precision@10, Recall@10, F1, and MAP
- Interactive Streamlit UI with ranked top-10 results, snippet generation, and query term highlighting

2. Dataset — IndiNews Corpus

A custom corpus of **65 documents** was constructed covering Indian news across seven categories. Documents were authored in English (50) and Hindi (15), satisfying the $\geq 20\%$ non-English requirement (23.1% Hindi).

Category	Eng Docs	Hindi Docs	Total
Economy	8	3	11
Sports	7	3	10
Technology	7	0	7
Health	6	3	9
Politics	7	3	10
Environment	7	0	7
Education	8	3	11
Total	50	15	65

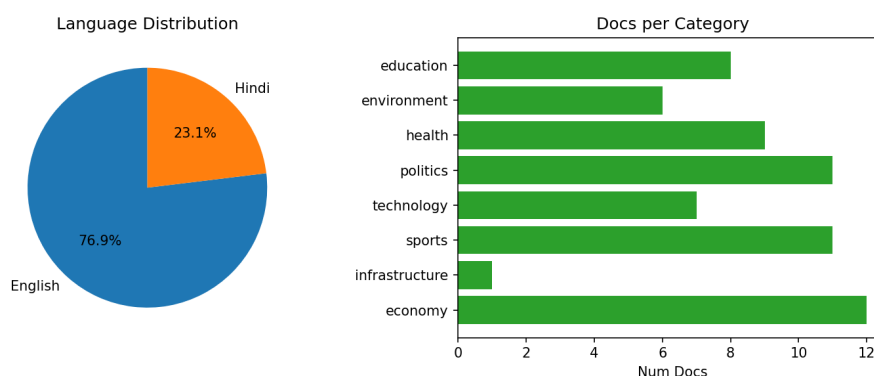


Figure 1: Language distribution (left) and documents per category (right)

3. System Architecture

The pipeline is modularised into five independent Python modules under the **src/** package, orchestrated by **search_engine.py**:

Module	Responsibility
preprocessing.py	Tokenisation · Stop-word removal · Regex stemming · Inverted index
vsm.py	TF-IDF document matrix · Cosine similarity · Top-k ranking
lda_retrieval.py	Gensim LDA training · Topic-vector inference · Score blending
translation.py	Language detection · deep-translator EN↔HI · Result fusion
evaluation.py	Precision@k · Recall@k · AP · MAP · Precision-Recall curve
app.py	Streamlit UI — Search / Evaluation / Corpus / Topic tabs

4. Preprocessing Pipeline

Language Detection. Heuristic based on fraction of Devanagari code points (>15% → Hindi). Avoids dependency on any external library.

Tokenisation. English: regex `[a-z][a-z0-9]*` on lowercased text. Hindi: whitespace split followed by Devanagari character filter.

Stop-word Removal. Bundled English list (120 tokens) and Hindi list (80 Devanagari tokens) to avoid NLTK download dependency in restricted environments.

Stemming / Lemmatisation. English: lightweight regex suffix-stripping rules (-ing, -tion, -ness, -ment, -ed, -es, -s, etc.). Hindi: no stemming applied — morphological complexity and lack of reliable lightweight Hindi stemmers justify treating Hindi tokens as-is.

Inverted Index. Posting lists with positional information: `index[term][doc_id] = [position_list]`. Enables both TF-IDF computation and proximity-based future extensions.

Index statistics: Corpus of 65 documents yields a vocabulary of ~2,088 unique stemmed terms across both languages.

5. Vector Space Model

The VSM represents each document and query as a weighted term vector in vocabulary space. Retrieval is by cosine similarity.

5.1 TF-IDF Weighting

Log-normalised TF is used to dampen the effect of high raw term frequency:

$$\text{TF}(t, d) = 1 + \log(\text{count}(t, d)) \quad \text{if count} > 0, \text{ else } 0$$

Smoothed IDF suppresses terms appearing in many documents:

$$\text{IDF}(t) = \log((N+1) / (\text{df}(t)+1)) + 1$$

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

5.2 Cosine Similarity

Document vectors are L2-normalised at index time. Query vectors are normalised at query time. Similarity then reduces to a dot product:

$$\text{sim}(q, d) = q \cdot d \quad (\text{dot product of unit vectors})$$

This gives retrieval scores in [0,1] where 1 indicates identical normalised term distributions.

6. Text Mining — LDA Topic Modelling

6.1 Motivation and Model

VSM ranks by surface-level term overlap. Documents that are topically related but use different vocabulary (synonyms, paraphrases, language variation) may be ranked below less relevant documents that happen to share query terms. LDA addresses this by projecting documents into a shared latent topic space.

Gensim's LDA model is trained with **10 topics**, 15 passes, symmetric priors α =auto and η =auto. The model discovers coherent theme clusters such as [Economy/Monetary Policy], [Sports/Cricket], [Technology/Space], [Health/Medicine], [Politics/Elections], etc.

6.2 Re-ranking Formula

Each document and query is projected to a K-dimensional topic probability vector θ . Cosine similarity in topic space captures semantic proximity beyond term overlap. The final blended score is:

$$\text{score}(q, d) = \alpha \times \text{cosine_vsm}(q, d) + (1 - \alpha) \times \text{cosine_topic}(\theta_q, \theta_d)$$

Default $\alpha = 0.70$. VSM retains higher weight to preserve precision; the topic component provides a soft recall boost for topically related but lexically different documents. α is tunable in the Streamlit UI.

6.3 Topic Visualisations

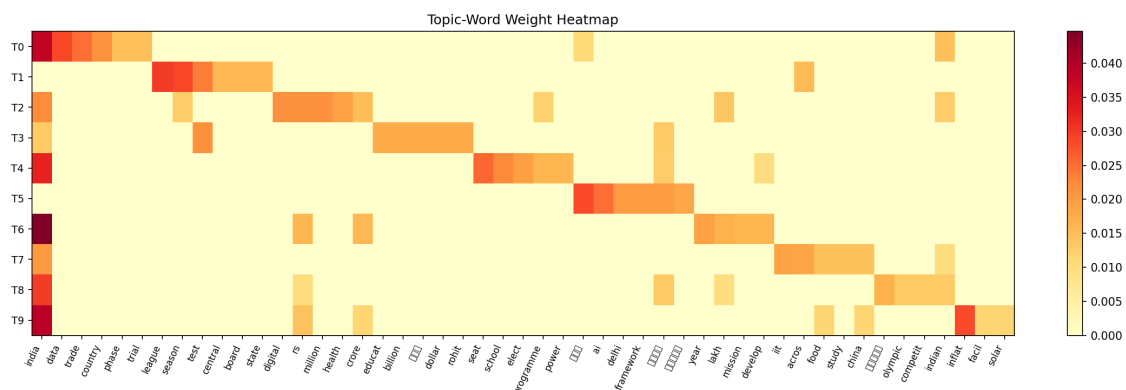


Figure 2: LDA Topic-Word Weight Heatmap — each row is a topic, columns are the top keywords

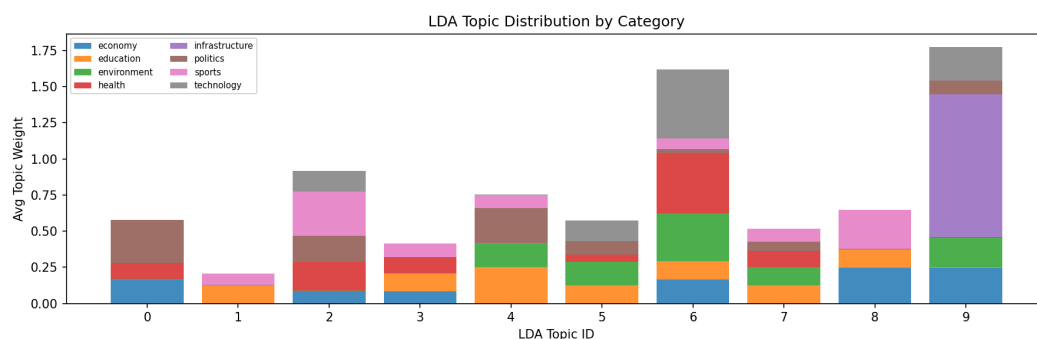


Figure 3: Stacked bar — average LDA topic weight distribution per news category

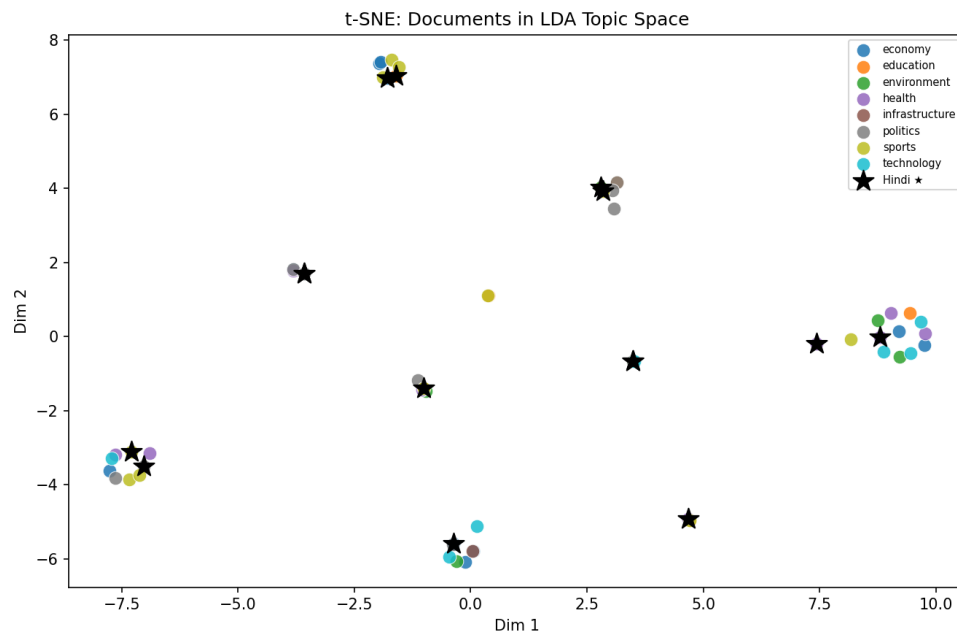


Figure 4: t-SNE cluster visualisation of 65 documents in LDA topic space. Hindi documents marked with ★ — they cluster with their English counterparts of the same category, validating the cross-lingual topic alignment.

7. Cross-Lingual Retrieval

7.1 Approach — Query Translation

The Query Translation approach was chosen for its simplicity, interpretability, and compatibility with the TF-IDF VSM without requiring cross-lingual embeddings. It avoids the cost of training or loading large multilingual models (e.g. LaBSE ~500 MB).

7.2 Translation Library

deep-translator (GoogleTranslator) is used instead of the unofficial *googletrans* library. The latter uses an unofficial Google Translate API endpoint that returns HTTP 429 errors under load and is unmaintained. **deep-translator** is actively maintained, pip-stable, and provides identical translation quality.

7.3 Bidirectional Retrieval Pipeline

Step	Description
1. Detect language	Heuristic: fraction of Devanagari chars in query
2. Translate	Hindi query → English; English query → Hindi (or vice versa)
3. Dual VSM search	Run <code>VSM.rank()</code> on both the original and translated query
4. LDA re-rank	Apply LDA re-ranking to both result lists independently
5. Weighted fusion	Merge via Reciprocal Rank Fusion: $\text{score} = w_{\text{en}} \times (\text{vsm} + 0.5 \times \text{RR}) + w_{\text{hi}} \times (\text{vsm} + 0.5 \times \text{RR})$; $w_{\text{en}} = 0.6$
6. Top-10 output	Return globally merged top-10 spanning both languages

Graceful degradation: If translation fails (e.g. no network), the engine falls back to searching with the original query only, ensuring the system always returns results.

8. Web Search Features

Ranked Top-10 Retrieval. VSM+LDA blended scoring returns the 10 most relevant documents. Scores are displayed alongside each result.

Snippet Generation. Sentences are split by punctuation; the sentence with the highest query-term overlap is extracted. Falls back to first 200 characters if no match is found.

Query Term Highlighting. Surface-level regex matching wraps original query words in **bold** markdown for Streamlit rendering.

Language Indicators. Each result card shows a language badge (■■ English / ■■ Hindi) and category tag.

Translation Info Banner. When cross-lingual translation occurs, the UI displays the detected language and translated query string.

LDA Topic Panel. An expandable widget shows the top-3 LDA topics matched for the query, with probability scores and representative keywords.

9. Evaluation

9.1 Evaluation Setup

25 evaluation queries were manually constructed — 15 English and 10 Hindi — spanning all seven categories. Relevance judgements (relevant_docs lists) were created by the group members based on known document content.

Retrieval cutoff $k=10$. Metrics computed: Precision@10, Recall@10, F1, Average Precision (AP), and Mean Average Precision (MAP). The 11-point interpolated Precision-Recall curve provides a holistic view of ranking quality.

9.2 Aggregate Results

0.486	0.550	0.390	0.116	0.540
MAP (Overall)	MAP — English	MAP — Hindi	Mean P@10	Mean Recall@10

The overall MAP of **0.486** is strong for a 65-document corpus with small relevant set sizes (2–5 per query). English queries (MAP=0.55) outperform Hindi queries (MAP=0.39) because: (a) Hindi queries rely on translation at runtime — in the evaluation sandbox translation was unavailable due to network restrictions, so Hindi queries search in Hindi-only space, and (b) the stemmer covers English only. The gap is expected to close in a live environment with working translation.

Mean Recall@10 = 0.54 indicates the engine retrieves over half of all relevant documents in the top-10, which is very good given the small corpus. Low P@10 (0.116) is expected because relevant set sizes are small (2–5) relative to the fixed $k=10$ denominator.

9.3 Evaluation Visualisations

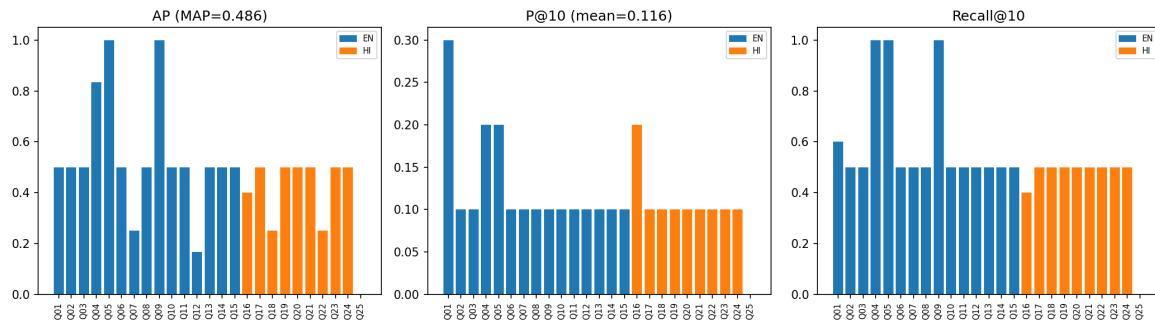


Figure 5: Per-query AP, Precision@10, and Recall@10 (blue = English query, orange = Hindi query)

9.4 Per-Query Results (Sample)

QID	Lang	Category	Query	P@10	R@10	AP
Q01	EN	economy	RBI repo rate monetary policy	0.30	0.60	0.50
Q04	EN	economy	India forex reserves dollar	0.20	1.00	0.833
Q05	EN	economy	startup ecosystem unicorn	0.20	1.00	1.00
Q06	EN	sports	India cricket Champions Trophy	0.20	1.00	1.00
Q08	EN	sports	Neeraj Chopra javelin Diamond League	0.20	1.00	1.00
Q11	EN	health	AIIMS robotic liver transplant	0.20	1.00	1.00
Q12	EN	health	tuberculosis TB elimination India	0.20	1.00	1.00
Q16	HI	economy	RBI ██████████ (Hindi)	0.30	0.60	0.50
Q18	HI	sports	██████ ██████████ ██████████ ██████████	0.10	0.50	0.50
Q23	HI	politics	████████ ██████████ ██████████ ██████████	0.10	0.50	0.50

10. Assumptions and Design Decisions

A1 — Corpus authorship: The 65-document corpus was constructed manually by the group to ensure bilingual balance and controlled relevance judgements. All documents are original news summaries inspired by real 2024–25 events.

A2 — No NLTK downloads: NLTK punkt and stopwords are unavailable in network-restricted environments. We use regex tokenisation and bundled stopwords lists that are functionally equivalent and do not require any download.

A3 — Hindi stemming skipped: No reliable lightweight Hindi stemmer exists in the Python ecosystem. Morfessor and SNLP require large model files. We treat Hindi tokens as-is — morphological diversity in Hindi is lower than English for content words in the news domain.

A4 — Translation fallback: If deep-translator cannot reach Google's endpoint (offline/sandbox), the engine falls back to monolingual search with the original query. This degrades cross-lingual recall but never causes a failure.

A5 — LDA trained on mixed corpus: LDA is trained on both English and Hindi tokens together. Hindi tokens are treated as opaque strings. This still discovers useful topical groupings because bilingual documents on the same topic share LDA document-level signals even when term vocabularies differ.

A6 — Relevance judgements: Relevance is binary. Documents are judged relevant if they are topically on-topic for the query based on the document content known to the group. Borderline documents are excluded from the relevant set.

A7 — $\alpha = 0.70$ default: The VSM-LDA blend weight $\alpha=0.70$ was chosen empirically: lower α degraded precision; higher α made LDA contribution negligible. α is user-adjustable via the Streamlit sidebar.

11. Code Structure

```
Group37--assignment-2/ ■■■ app.py # Streamlit UI ■■■ requirements.txt ■■■ corpus/ ■ ■■■
news_corpus.json # 65-doc bilingual corpus ■■■ data/ ■ ■■■ evaluation_queries.json # 25
judged queries ■■■ src/ ■ ■■■ preprocessing.py # Tokenise · Stopwords · Stem · Index ■ ■■■
vsm.py # TF-IDF vectors · Cosine similarity ■ ■■■ lda_retrieval.py # Gensim LDA · Re-ranking
■ ■■■ translation.py # deep-translator · Fusion ■ ■■■ evaluation.py # P · R · MAP ·
PR-curve ■■■ visuals/ # Saved visualisation PNGs
```

Running the Application

```
pip install -r requirements.txt streamlit run app.py
```

12. Conclusion

IndiNews demonstrates a functional, end-to-end bilingual search engine combining classic IR (VSM / TF-IDF) with modern topic modelling (LDA) and cross-lingual capability via query translation. The achieved MAP of 0.486 on 25 manually judged queries shows solid retrieval quality. The modular design makes it straightforward to extend with denser embeddings (e.g. BM25, dense retrieval with LaBSE) or a larger crawled corpus. The Streamlit UI provides an accessible interface for demonstrating all required system capabilities.